

Optimizing Layout of Recursive Data Types with Marmoset

Or, Algorithms + Data Layouts = Efficient Programs

ANONYMOUS AUTHOR(S)

While programmers know that the low-level memory representation of data structures can have significant effects on performance, compiler support to *optimize* the layout of those structures is an under-explored field. Prior work has optimized the layout of individual, *non-recursive* structures without considering how collections of those objects in linked or *recursive* data structures is laid out.

This work introduces MARMOSSET, a compiler that optimizes the layouts of algebraic data types, with a special focus on producing highly optimized, *packed* data layouts where recursive structures can be traversed with minimal pointer chasing. MARMOSSET performs an analysis of how a recursive ADT is used across functions to choose a *global* layout that promotes simple, strided access for that ADT in memory. It does so by building and solving a constraint system to minimize an abstract cost model, yielding a predicted efficient layout for the ADT. MARMOSSET then builds on top of GIBBON, a prior compiler for packed, mostly-serial representations, to synthesize optimized ADTs. We show experimentally that MARMOSSET is able to choose optimal layouts across a series of microbenchmarks and case studies, outperforming both GIBBON's baseline approach, as well as the mature GHC compiler with its traditional, pointer-heavy, non-compact data representations.

1 INTRODUCTION

Recursive data structures are readily available in most programming languages. Linked lists, search trees, tries and others provide efficient and flexible solutions to a wide class of problems—both in low-level languages with direct memory access (C, C++, Rust, Zig) as well as high-level ones (Java, C#, Python). Additionally, in the purely functional (or, persistent [Okasaki 1998]) setting, recursive, tree-like data structures largely replace array-based ones. Implementation details of recursive data structures are not necessarily known to application programmers, who can only hope that the library authors and the compiler optimize extensively to achieve good performance. Sadly, recursive data structures are a hard optimization target.

High-level languages represent recursive structures with pointers to small objects allocated sparsely on the heap. An algorithm processing such a *boxed* representation, a traversal, spends much time in pointer chasing. Pointer chasing is one of the most painful operations for modern hardware architectures. Optimizing compilers for these languages and architectures have many strengths but optimizing memory representation of user-defined data structures is not among them. One alternative is resorting to manual memory management to achieve maximum performance, but it has the obvious drawback of leaving convenience and safety behind. Is it the only alternative?

A radically different approach to representing recursive datatypes as dense structures (basically, arrays) has been studied recently. The GIBBON compiler tries to improve the performance of recursive data structures by embracing dense representations by default [Vollmer et al. 2017]. This choice has practical benefits for programmers: they no longer need to take control of low-level data representation and allocation to serialize linked structures; and rather than employing error-prone index arithmetic to access data, they let GIBBON automatically translate idiomatic data structure accesses into operations on the dense representation.

Dense representations are not a panacea, though. They can suffer a complementary problem due to their inflexibility. A particular serialization decision for a data structure made by the compiler can misalign with the behavior of functions accessing that data. Consider a tree laid out in left-to-right pre-order with a program that accesses that tree right-to-left. Rather than scanning

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

2018. ACM ISBN 978-1-4503-XXXX-X/18/06...\$15.00

<https://doi.org/XXXXXXX.XXXXXXX>

straightforwardly through the structure, the program would have to jump back and forth through the buffer to access the necessary data.

One way to counter the inflexibility of dense representations is to introduce some pointers. For instance, GIBBON inserts *shortcut pointers* to allow random access to recursive structures [Vollmer et al. 2019]. But this defeats the purpose of a dense representation: not only are accesses no longer nicely strided through memory, but the pointers and pointer chasing of boxed data are back. Indeed, when GIBBON is presented with a program whose access patterns do not match the chosen data layout, the generated code can be *significantly slower* than a program with favorable access patterns.

Are we stuck with pointer chasing when processing recursive data structures? We present MARMOSET as a counter example. MARMOSET is our program analysis and transformation approach that spots misalignments of algorithms and data layouts and fixes them where possible. Thus, our motto is:

Algorithms + Data Layouts = Efficient Programs

MARMOSET analyzes the data access patterns of a program and synthesizes a data layout that corresponds to that behavior. It then rewrites the datatype and code to produce more efficient code that operates on a dense data representation in a way that matches access patterns. This co-optimization of datatype and code results in improved locality and, in the context of GIBBON, avoidance of shortcut pointers as much as possible.

We implement MARMOSET as an extension to GIBBON—a mature compiler based on dense representations of datatypes. That way, MARMOSET can be either a transparent compiler optimization, or semi-automated tool for exploring different layouts during the programmer’s optimization work. Our approach has general applicability because of the minimal and common nature of the core language: the core language of MARMOSET (Figure 6) is a simple first-order, monomorphic, strict, purely functional language. Thanks to this design, we manage to isolate MARMOSET from GIBBON-specific, complicated (backend) mechanics of converting a program to operate on dense rather than boxed data.

Overall, in this paper:

- We provide a static analysis capturing the temporal access patterns of a function towards a datatype it processes. As a result of the analysis, we generate a *field access graph* that summarizes these patterns.
- We define a cost model that, together with the field access graph, enables formulating the field-ordering optimization problem as an integer linear program. We apply a linear solver to the problem and obtain optimal positions of fields in the datatype definition relative to the cost model.
- We extend the GIBBON compiler to synthesize new datatypes based on the solution to the optimization problem, and transform the program to use these new, optimized types, adjusting the code where necessary.
- Using a series of benchmarks, we show that our implementation, MARMOSET, can provide speedups of 1.14 to 54 times over the best prior work on dense representations, GIBBON. MARMOSET outperforms GHC on these same benchmarks by a factor of 3 to 6, when compiling in strict (call-by-value) mode.

2 DENSE REPRESENTATION: THE GOOD, THE BAD, AND THE POINTERS

Algebraic datatypes (ADTs) are a powerful language-based technology. ADTs can express many complex data structures while, nevertheless, providing a pleasantly high level of abstraction for application programmers. The high-level specification of ADTs leaves space to experiment with

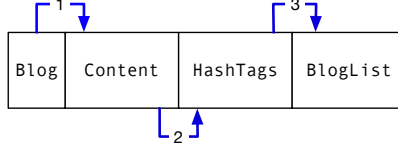


Fig. 1. Dense pointer free layout with ideal access patterns.

low-level implementation strategies. Hence, we use ADTs and a purely functional setting for our exploration of performance implications of data layout.

In this section, we use an example to illustrate the performance implications of picking a layout for a recursive data structure with a dense memory representation. In particular, consider a datatype (already monomorphized) describing a sequence of posts in a blog¹:

```
data BlogList = Nil | Blog Content HashTags BlogList
```

A non-empty blog value stores a content field (a string representing the body of the blog post), a list of hash tags summarizing keywords of the blog post, and a pointer to the rest of the list.²

In a conventional implementation of algebraic datatypes, accessing a value of a given ADT requires dereferencing a pointer to a heap object, then reading the header word, to get to the payload. Accessing the desired data may require multiple further pointer dereferences, as objects may contain pointers to other objects, requiring the unraveling of multiple layers of nesting.

The whole process is often described as *pointer chasing*, a fitting name, especially when the work per payload element is low.

In a dense representation of ADTs, as implemented in GIBBON, the data constructor stores one byte for the constructor's *tag*, followed immediately by its fields. Wherever possible, the tag value occurs inline in a bytestream that hosts multiple values. As a result, values tend to reside compactly in the heap using contiguous blocks of memory. This representation avoids or reduces pointer chasing and admits efficient linear traversals favored by modern hardware via prefetching and caching.

The efficiency of traversals on dense representations of data structures largely depends on how well access patterns and layout match each other. Going back to the example, consider the most favorable case first: assume that every function accessing a blog value accesses the fields in the same order. Moreover, suppose that the order of the accesses is the same as the order in which the fields appear in the datatype definition. In this case, GIBBON can assign the dense and pointer-free layout as shown in Figure 1. Solid blue arrows connecting adjacent fields represent unconditional sequential accesses—i.e., reading a range of bytes in a buffer, and then reading the next consecutive range. Such a traversal will reap the benefits of locality.

On the other hand, consider a traversal with less efficient access patterns (Figure 2). The algorithm scans blog entries for a given keyword in `HashTags`. If the hash tags of a particular blog entry contain the keyword, the algorithm puts an emphasis on every occurrence of the keyword in the content field. In terms of access patterns, if we found a match in the hash tags field, subsequent accesses to

¹Throughout the paper we use a subset of Haskell syntax, which corresponds to the input language of the GIBBON compiler.

²In practice, you may want to reuse a standard list type, e.g.:

```
type BlogList = [Blog]
data Blog = Blog Content HashTags
```

but a typical compiler (including GIBBON) would specialize the parametric list type with the `Blog` type to arrive at an equivalent of the definition shown in the main text.

```

148 emphKeyword :: String → BlogList → BlogList
149 emphKeyword keyword blogs = case blogs of
150   Nil → Nil
151   Blog content hashTags blogs' →
152     if search keyword hashTags then
153       let content' = emphContent content keyword
154         blogs'' = emphKeyword keyword blogs'
155       in Blog content' hashTags blogs''
156   else
157     let blogs'' = emphKeyword keyword blogs'
158     in Blog content hashTags blogs''

```

Fig. 2. Blog traversal motivating example

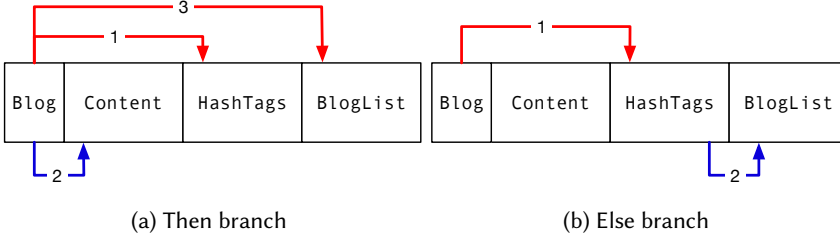


Fig. 3. Dummy traversals to access fields out of order.

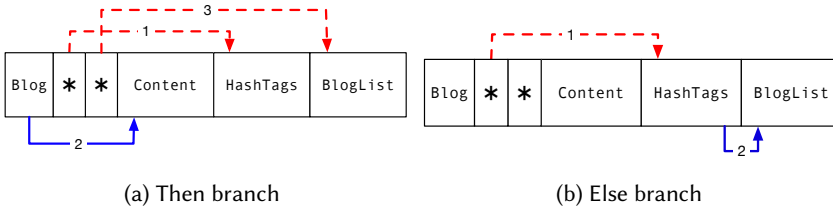


Fig. 4. Representation with pointers to allow random access.

the fields happen in order, as depicted in Figure 3a. Otherwise, the traversal skips over the content field, as depicted in Figure 3b.

Here we use red lines to represent accesses that must *skip over* some data between the current position in the buffer and the target data. Data may be constructed recursively and will not necessarily have a statically-known size, so finding the end of a piece of unneeded requires scanning through that data in order to reach the target data. This “dummy” traversal requires an arbitrarily large amount of work because the content field has a variable, dynamically-allocated size.

Dummy traversals, like skipping over the content field above, can degrade the asymptotic efficiency of programs. One way to avoid such traversals is to use pointers. For instance, when GIBBON detects that has to skip over intervening data, it changes the definition of the constructor by inserting *shortcut pointers*, which provide an exact memory address to skip to in constant time. For our example program, GIBBON introduces one shortcut pointer for the `HashTags` field, and another one for the tail of the list.

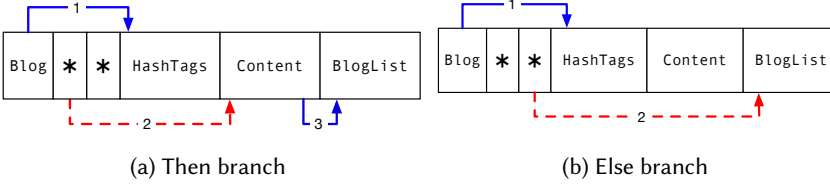


Fig. 5. Optimized layout with favorable access patterns.

The pointers provide direct accesses to the respective fields when needed and restore the constant-time asymptotic complexity for certain operations. This results in the access patterns shown in Figure 4. Red dashed lines represent pointer-based constant-time field accesses. Otherwise, the access patterns are similar to what we had before introducing the pointers.

The pointer-based approach in our example has two weaknesses. First, this approach is susceptible to the usual problems with pointer chasing. Second, just like with the initial solution, we access fields in an order that does not match the layout: the hash tags field is always accessed first but lives next to the content field.

MARMOSET, described in the following section, automates the process of finding the weaknesses of the pointer-based approach and improving the layout and code accordingly. For instance, performance in our example can be improved by swapping the ordering between the `Content` and `HashTags` fields. Given this reordering, the hash tags are available directly at the start of the value, which lines up with the algorithm better, as the algorithm always accesses this field first. Additionally, our program’s “then” branch (the keyword gets a hit within the hash tags) is more efficient because after traversing the content to highlight the keyword it stops at the next blog entry ready for the algorithm to make the recursive call. This improved data layout results in the more-streamlined access patterns shown in Figure 5.

3 DESIGN

MARMOSET infers efficient layouts for dense representations of recursive datatypes. MARMOSET’s key idea is that the best data layout should match the way a program accesses these data. Section 2 shows how this idea reduces to finding an ordering of fields in data constructors. The ordering must align with the order a function accesses those fields, in which case the optimization improves performance of the function.

To find a better layout for a datatype in the single-function case, MARMOSET first analyzes possible executions of the function and their potential for field accesses. In particular, MARMOSET takes into account (a) the various paths through a function, each of which may access fields in a different order, and (b) dependencies between operations in the function, as in the absence of dependencies, the function can be rewritten to access fields in the original order, and that order will work best. MARMOSET thus constructs a control flow graph (Section 3.2) and collects dataflow information (Section 3.3) to build a *field access graph*, a representation of the various possible orders in which a function might access fields (Section 3.4 and Section 3.5).

Once data accesses in a function are summarized in the field access graph, MARMOSET proceeds with synthesizing a data layout. MARMOSET incorporates knowledge about the benefits of sequential, strided access and the drawbacks of pointer chasing and backtracking to define an abstract cost model. The cost model allows to formulate an integer linear program whose optimal solution corresponds to a layout that minimizes the cost according to that model (Section 3.6).

$K \in$ Data Constructors, $\tau \in$ Type Constructors, $x, y, f \in$ Variables

Top-Level Programs	top	$::= \overline{dd}; \overline{fd}; e$
Field of a Data Constructor	fe	$::= \tau$
Datatype Declarations	dd	$::= \text{data } \tau = \overline{K \overline{fe}}$
Function Declarations	fd	$::= f : \overline{\tau} \rightarrow \tau; f \overline{x} = e$
Values	v	$::= x$
Expressions	e	$::= v$ $ f \overline{v}$ $ K \overline{v}$ $ \text{let } x : \tau = e \text{ in } e$ $ \text{case } v \text{ of } \overline{pat}$
Pattern	pat	$::= K (\overline{x} : \overline{\tau}) \rightarrow e$

Fig. 6. Simplified grammar for a first-order, monomorphic functional language that MARMOSSET operates on. The full implementation includes more primitive datatypes, and built in primitives (such as a copy primitive).

The remainder of this section walks through this design in detail, and discusses how to extend the system to handle multiple functions that use a datatype (Section 3.7).

3.1 MARMOSSET's Language

MARMOSSET operates on the language λ_M given in Figure 6. λ_M is a first-order, monomorphic, call-by-value functional language with algebraic datatypes and pattern matching. Programs consist of a series of datatype definitions, function definitions, and a main expression. λ_M 's expressions use A-normal form [Flanagan et al. 1993]. The notation $\overline{x}$ denotes a vector $[x_1, \dots, x_n]$ and $\overline{x}_i$ the item at position i . λ_M is an intermediate representation (IR) used towards the front end in the Gibbon compiler. The monomorphizor and specializer lower a program written in polymorphic, higher-order subset of Haskell³ to λ_M , and then location inference is used to convert it to the *location calculus* (LoCal) code next [Vollmer et al. 2019]. It is easier to update the layout of types in λ_M compared to LoCal, as in λ_M the layout is implicitly determined by the ordering of fields, whereas the later LoCal IR makes the layout explicit using locations and regions (essentially, buffers and pointer arithmetic).

3.2 Control Flow Analysis

We construct a control flow graph with flattened, let-bound expressions of λ_M as the nodes. Edges between the nodes represent paths between expressions. The edges are labeled with *weights* that represent the likelihood of a particular path's being taken. Given a function definition, each sub-expression of the function body becomes a separate node in the CFG. An edge between two nodes indicates the order of the evaluation of the program⁴. A node corresponding to a *let*-binding contains the bound expression and has one outgoing edge to a node corresponding to the body expression. A *case* expression splits the control flow n -ways, where n is the number of pattern matches. Outgoing edges of a node for a *case* expression have weights associated with them that correspond to the likelihood of taking a particular branch in the program. If available, dynamic

³With strict evaluation semantics using -XStrict.

⁴The order of evaluation is straightforward to determine since λ_M uses call-by-value semantics.

profiling information can provide these weights. For the purposes of this paper, however, we find it is sufficient to use trivial placeholder weights: i.e. uniform weight over all branches. A node for a pattern match clause merely has an outgoing edge to its body expression. Control flow terminates on either a data constructor, a function application, or a variable reference.

Figure 7 shows the control-flow graph for the running example from Figure 2. Each node corresponds to a sub-expression of the function `emphKeyword`. The first `case` expression splits the control flow into two branches, corresponding to whether the input list of blogs is empty or not. The branch corresponding to the empty input list is assigned a probability α , and thus, the other branch is assigned a probability $1 - \alpha$. The next node corresponds to the pattern match `Blog content hashTags blogs'`. Another two-way branch follows, corresponding to whether `keyword` occurs in the content of this blog or not. We assign the probabilities σ and $1 - \sigma$ to these branches respectively. Note that as a result, the corresponding edges in the CFG have weights $(1 - \alpha) * \sigma$ and $(1 - \alpha) * (1 - \sigma)$, as the likelihood of *reaching* that condition is $(1 - \alpha)$. Each of these branches terminate by creating a new blog entry with its content potentially updated. In the current model, α and σ are 0.5: they are uniformly distributed.

One intuition for why realistic branch weights are not essential to MARMOSET's optimization is that accurate weights only matter if there is a trade off between control-flow paths that are best served by different layouts. The base cases (e.g. empty list) typically contribute *no* ordering constraints, and in our experience, traversals tend to have a preferred order per function, rather than tradeoffs intra-function, which would reward having accurate, profile-driven branch probabilities. Hence, for now, we use uniform weights even when looking at the intra-function optimization.

3.3 Data Flow Analysis

We implement a straightforward analysis (use-def chain, and def-use chain) for `let` expressions to capture dependencies between `let` expressions. We use this dependence information to form dataflow edges in the field access graph (Section 3.5) and to subsequently optimize the layout and code of the traversal for performance. For *independent* `let` expressions in the function we are optimizing, we can transform the function body to have these `let` expressions in a different order. (Independent implies that there are no data dependencies between such `let` expressions. Changing the order of independent `let` expressions will not affect the correctness of the code, as there are no exceptions in this pure context.) However, we only do such a transformation when we deem it to be more cost efficient. In order to determine when re-ordering `let` expressions is more cost efficient, we classify fields based on specific attributes next.

3.4 Attributes for fields to analyze the cost of different orderings of the layout and code

In section 2, we described the process to re-order fields of a data constructor by matching the layout to access patterns of the traversal. In this process, we treat the code as rigid and the layout as a free variable. However, it may be possible to change the accesses a traversal induces if there is no data dependence that guards that access.

In traversal 2, the `HashTags` field induces a data dependence through the `if` statement caused by the call to `search`. For that reason, we cannot change the code that causes the access to the `HashTags` field. However, in the `then` branch, the `let` expressions that cause the access to the content and tail of the list are not bound by any data dependence. We cannot move them out of the `then` branch because of the data dependence from `HashTags`, however, within the `then` branch we can re-order the `let` expressions that access the content and the tail of the list respectively.

Which order is better: to recurse on the tail of the list first and then emphasize content or vice-versa? To find out, we classify each field based on a few attributes. Attributes for each field are classified as belonging to either *recursive*, *scalar*, *self-recursive*, or *inlineable*. A field is *recursive*

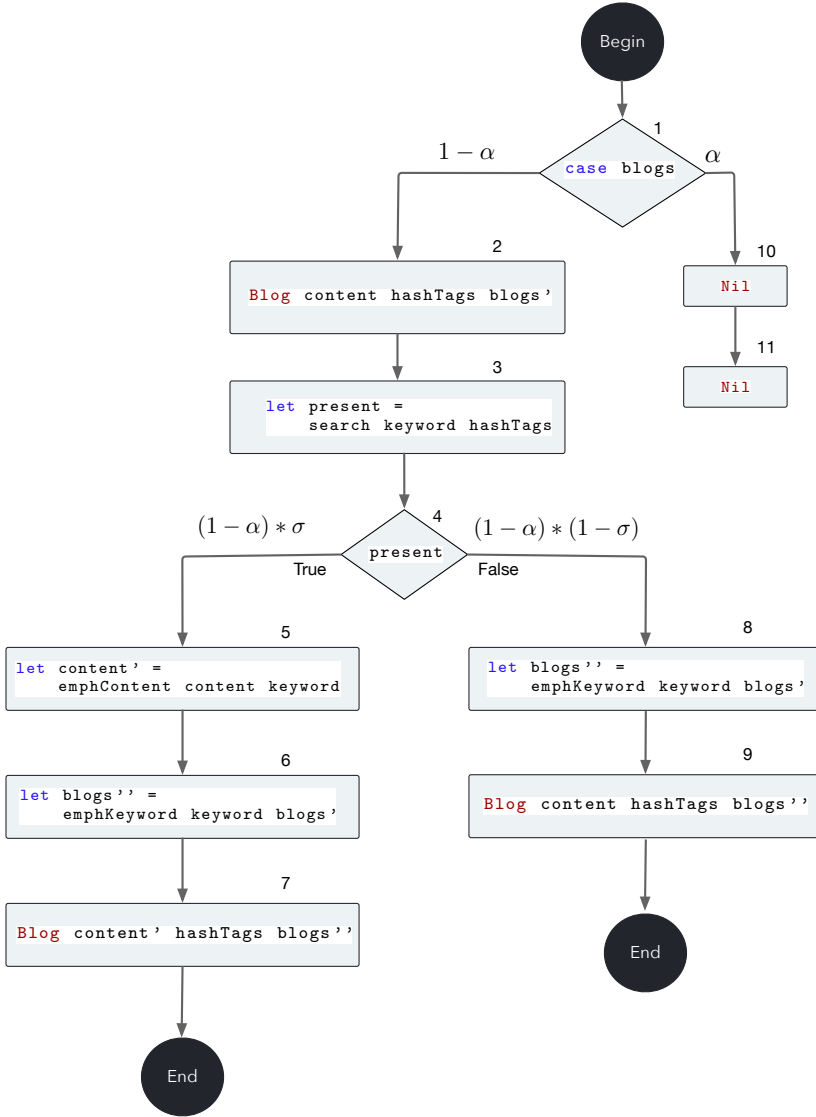


Fig. 7. Control flow graph generated from function body of running example 2.

if it refers to a datatype defined recursively. A field is *self-recursive* if it is a recursive field that directly refers back to the datatype being defined (such as a `List` directly referencing itself). A *scalar* field (non-recursive) refers to a datatype only made up of either other primitive types or datatypes containing only scalar fields (such as `Int`). Finally, we call a field *inlineable* if the function being optimized makes a recursive call into this field (i.e. taking the field value as an argument). Hence, an inlineable field is obviously a recursive field. The inlineable attribute is important when choosing whether to do code-motion or not to. A field has this attribute based on the scope (that is, how it is used in a function.) In the running example, the `BlogList` field is an inlineable field.

For each function in the program, we introduce a new metadata field that stores the attributes for all fields of all data constructors used in that function. A single field can have multiple attributes. For instance, A function (`traverse`) doing a pre-order traversal on a `Tree` data type makes recursive calls on both the left and right children. The left and right children are recursive and self-recursive. Therefore, when looking at the scope of `traverse`, the left and right children have the attributes recursive, self-recursive and inlineable. Consider the example of a list traversal:

```
foo :: List → List
foo lst = case lst of
  Nil → Nil
  Cons x rst →
    let x' = x ^ 100
        rst' = foo rst
    in Cons x' rst'
```

Here, the function `foo` does work on the `Int` field. It raises the `Int` to the power of 100, then recurs on the tail of the list. Note that MARMOSET is compiling these datatypes to be packed. This implies that the `List` type is going to be packed in memory with a representation that stores the `Cons` tag (one byte) followed by the `Int` (8 bytes) followed by the next `Cons` tag and so on. Hence, the `Cons` tag and the `Int` field are interleaved together in memory. The function `foo` becomes a stream processor that consumes one stream in memory and produces a dense output buffer of the same type.

On the other hand, the other way to lay out the list is to change its representation to the following.

```
data List' = Nil | Cons List' Int
```

In memory, the list has all `Cons` tags next to each other (a unary encoding of array length!) and the `Int` elements all next to each other. In such a scenario, we can express the expected performance of our traversal `foo` on the `List` type as a function of the work done and the size of the element field. As the work done on the element field increases or the length of the field itself increases the cost to traverse the field reduces in the second representation. Because fields are packed next to each other. Such a packing has better locality and the potential to reduce the runtime of the traversal `foo`. However, this only works if we can subsequently change the function that traverses the list to do recursion on the tail of the list first and then call the exponentiation function on the `Int` field after the recursive call. If there are no data dependencies between the recursive call and the exponentiation function, then this is straightforward. We show `foo'` with the required code motion transformation to function `foo` accompanied with the change in the data representation from `List` to `List'`.

```
foo' :: List' → List'
foo' lst = case lst of
  Nil → Nil
  Cons rst x →
    let rst' = foo' rst
        x' = x ^ 100
    in Cons rst' x'
```

We argue, that this transformation is similar to inlining the recursive call and changing the representation of `List` to a *structure of arrays* optimization which also causes all the same types to be next to each other in memory (i.e. alternating tag/int/tag/int values are changed to repeated tag/tag/int/int).

To optimize the layout, the tail of the list is assigned the attribute of inlineable. This attribute is used by the solver to determine a least-cost ordering to the `List` data type in the scope of function `foo`. Whenever such code motion is possible, MARMOSET will place the inlineable field first and use code-motion to change the body of the function to perform recursion first if data flow dependencies

Table 1. Running times for two traversals working with their corresponding matching layouts.

datatype / traversal	List / foo	List' / foo'
1M elements	0.992s	0.953s

allow such a transformation. In fact, we made a similar program and ran MARMOSSET on the program in the hope that it would be able to optimize the program by placing the inlineable field first and use code-motion to change the traversal. We show the results in table 1. We see a speedup of 4.1% over our baseline datatype and traversal. Note that the traversals `foo` and `foo'` have access patterns that are completely aligned with the data layout of `List` and `List'` respectively. The resulting speedup is solely a consequence of the *structure of arrays* effect. This is an added benefit to the runtime in addition to ensuring that the access patterns of a traversal are aligned with the data layout of the data type it traverses. For the running example, MARMOSSET chooses a layout where `HashTags` is followed by the tail pointer `BlogList` and `Content` respectively.

```
data BlogList = Nil | Blog HashTags BlogList Content
```

The traversal also changes such that the recursive call to the tail of the list `BlogList` occurs first, followed by the call to emphasize the `Content` in the `then` branch. This layout and traversal transformation provides the best performance.

3.5 Field Access Pattern Analysis

After constructing the CFG and DFG for a function definition, we inspect the type of each input parameter—one data constructor at a time—and construct a *field-access graph* for it. This graph represents the temporal ordering of accesses among its fields according to the function's CFG and DFG. The fields of the data constructor form the nodes of this graph. A directed edge from field f_i to field f_j is added if f_i is accessed immediately before f_j . A directed edge can be of two different types. In addition, each edge has a associated weight which indicates the likelihood of accessing f_i before f_j , which is computed using the CFG. An edge can either be a data flow edge or a control flow edge. In Figure 8, the red edge is a data flow edge and the blue edge is a control flow edge.

Data Flow Edge indicates an access resulting from a data flow dependence between the fields f_i and f_j . In our source language, a data flow edge is induced by either an `if` conditional or a `case` expression. A data flow edge implies that the code that represents the access is rigid in structure and changing it can make our transformation invalid.

Control Flow Edge indicates an access that is not data-flow dependent. It is caused by the control flow of the program. Such an edge does not induce strict constraints on the code that induces the edge. The code is malleable in case of such accesses. This gives way to an optimization search space via code motion of let expressions. The optimization search space involves transformation of the source code, i.e, changing the access patterns at the source code level.

The field-access graph G is a directed graph, which consists of edges of the two types between fields of a datatype and can have cycles. The directed nature of the edges enforces a temporal relation between the corresponding fields. More concretely, assume that an edge e' that connects two vertices representing fields fe_a (source of e') and fe_b (target of e'). We interpret e' as a temporal access such that field fe_a is accessed before field fe_b . The weight w' for the edge e' is the probability that this access will happen based on statically analyzing a function.

In our analysis, for a unique path through the traversal, we only account for the *first* access to any two fields. If two fields are accessed in a different order later on, the assumption is that the start address of the fields is likely to be in cache and hence it does not incur an expensive fetch call

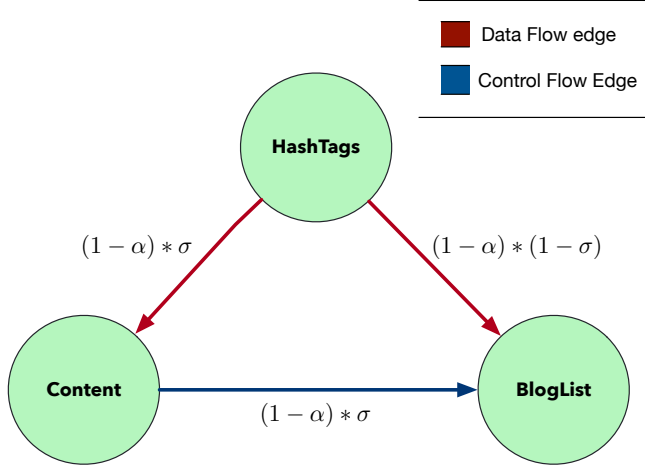


Fig. 8. Field access graph G generated from CFG of running example 7. Here we use field types to uniquely name them, since Haskell record syntax was not used in this example to give each field a name.

to memory. In fact, we tested our hypothesis by artificially making an example where say field f_a is accessed first, field f_b is accessed after f_a after which we constructed multiple artificial access edges from f_b to f_a , which might seem to suggest placing f_b before f_a . However, once the cache got warmed up and start addresses of f_a and f_b are already in cache, the layout did not matter as much. This suggests that prioritizing for the first access edge between two unique fields along a unique path is sufficient for our analysis.

Two fields can be accessed in a different order along different paths through a traversal. This results in two edges between the fields. (The edges are in reversed order.) We allow a max of two edges between any two vertices with the constraint that they have to be in the opposite direction and come from different paths in the traversal. If two fields are accessed in the same order along different paths in the traversal, we simply add the probabilities and merge the edges since they are in the same direction.

In order to construct G , we topologically sort the control flow graph of a function and traverse it in the breadth-first fashion. We maintain an environment that tracks aliases to fields in the data constructor for which we are constructing the field-access graph G . As we process each node (i.e. a primitive expression such as a single function call), we update the graph for any direct or indirect references to input fields that we can detect. We ignore new variable bindings that refer to newly allocated rather than input data—they are not tracked in the access graph. We traverse the control-flow graph once, but we maintain the last-accessed information at each CFG node, so when we process a field access at an expression, we consult what was previously-accessed at the unique predecessor of the current CFG node. Figure 8 shows the generated access graph from the control flow graph in Figure 7. It also shows the probability along each edge obtained from the control flow graph.

As we are traversing the nodes of the control flow graph and generating directed edges in G , we also keep track of the weights for each edge and store the weights appropriately. As mentioned previously, we can get more precise weights by dynamically profiling the traversal at runtime to obtain the frequency of taking a particular path through the traversal. However, for now, we restrict ourselves to static weights.

3.6 Finding a Layout

We use the field-access graph G to encode the problem of finding a better layout as an Integer Linear Program (ILP). Solving the problem yields a cost-optimal field order for the given pair of a data constructor and a function.

3.6.1 ILP Constraints. In our encoding, each field in the data constructor is represented by a variable, $f_0, f_1, \dots$. As a part of the result, each variable will be assigned a unique integer in the interval $[0, n - 1]$, $n = \text{length}(\text{fields})$. Intuitively, each variable represents an index in the sequence of fields.

The ILP uses several forms of constraints, including two forms of *hard* constraints:

$$\forall_{0 \leq i < n} \quad 0 \leq f_i < n \quad (1)$$

$$\forall_{0 \leq i < j < n} \quad f_i \neq f_j \quad (2)$$

The constraints of form 1 ensure that each field is mapped to a valid index, while the constraints of form 2 ensure that each field has a unique index. Constraints of either form must hold because each field must be in a valid location.

Hard constraints define valid field orderings but not all such reorderings improve efficiency, MARMOSET's main goal. To fulfil the goal, beside the hard constraints we introduce *soft* ones. Soft constraints come from the field access analysis. For example, assume that based on the access pattern of a function, we would *prefer* that field a goes before field b . We turn such a wish into a constraint. If the constraint cannot be satisfied, it will not break the correctness. In other words, such constraints can be broken, and that is why we call them soft.

3.6.2 COST MODEL. MARMOSET encodes these soft constraints in the form of an abstract cost model that assigns a cost to a given layout (assignment of fields to positions) based on how efficient it is expected to be given the field-access graph.

To understand the intuition behind the cost model, note that the existence of an edge from field f_i to field f_j in the field-access graph means that there exists at least one path in the control-flow graph where f_i is accessed and f_j is the next field of the data constructor that is accessed. In other words, the existence of such an edge implies a preference *for that control flow path* for field f_i to be immediately before field f_j in the layout so that the program can continue a linear scan through the packed buffer. Failing that, it would be preferable for f_j to be "ahead" of f_i in the layout so the program does not have to backtrack in the buffer. We can thus consider the costs of the different layout possibilities of f_i and f_j :

C_{succ} (f_j immediately after f_i): This is the best case scenario: the program traverses f_i and then uses f_j .

C_{after} (f_j after f_i in the buffer): If f_j is after f_i , but not *immediately* after, then the code can proceed without backtracking through the buffer, but the intervening data means that either a shortcut pointer or a dummy traversal must be used to reach f_j , adding overhead.

C_{pred} (f_j immediately before f_i): Here, f_j is *earlier* than f_i in the buffer. Thus, the program will have already skipped past f_j , and some backtracking will be necessary to reach it. This incurs *two* sources of overhead: skipping past f_j in the first place, and then backtracking to reach it again.

C_{before} (f_j before f_i in the buffer): If, instead, f_j is farther back in the buffer than f_i , then the cost of skipping back and forth is greater: in addition to the costs of pointer dereferencing, because the fields are far apart in the buffer, it is less likely f_j will have remained in cache (due to poorer spatial locality).

We note a few things. First, the *exact* values of each of these costs are hard to predict. The exact penalty a program would pay for jumping ahead or backtracking depends on a variety of factors such as cache sizes, number of registers, cache line sizes, etc.

However, we use our best intuition to statically predict these costs based on the previously generated access graph. Note the existence of two types of edges in our access graph. An edge can either be a data flow edge or a control flow edge. For a data flow edge, the code is rigid. Hence the only axis we have available for transformation is the datatype itself. For a data flow edge, the costs are showed in Eq 3. Here, we must respect the access patterns in the original code which lead to the costs in Eq 3.

$$C_{succ} < C_{after} < C_{pred} < C_{before} \quad (3)$$

Note that a control flow edge signifies that the direction of access for an edge is transformable. We could reverse the access in the code without breaking the correctness of the code. We need to make a more *fine-grained* choice. This choice involves looking at the attributes of the fields and making a judgement about the costs given we know the attributes of the fields. As shown in Sec 3.4 we would like to have the field with an *inlineable* attribute placed first. Hence, in our cost model, if f_i is inlineable, then we follow the same costs in Eq 3. However, if f_j is inlineable and f_i is not. We would like f_j to be placed before f_i . For such a layout to endure, the costs should change to Eq 4. For other permutations of the attributes, we use costs that prioritize placing the inlineable field/s first.

$$C_{pred} < C_{before} < C_{succ} < C_{after} \quad (4)$$

3.6.3 ASSIGNING COSTS TO EDGES. MARMOSSET uses the field-access graph and the cost model to construct an objective function for the ILP. Each edge in the access graph represents one pair of field accesses with a preferred order. Thus, for each edge $e = (i, j)$, MARMOSSET can use the indices of the fields f_i and f_j to assign a cost, c_e , to that pair of accesses.

If f_j is right after f_i , assign cost C_{succ} :

$$(f_j - f_i) = 1 \rightarrow c_e = C_{succ}$$

If f_j is farther ahead of f_i , assign cost C_{after} :

$$(f_j - f_i) > 1 \rightarrow c_e = C_{after}$$

If f_j is immediately *before* f_i , assign cost C_{pred} :

$$(f_j - f_i) = -1 \rightarrow c_e = C_{pred}$$

And if f_j is farther before f_i , assign cost C_{before} :

$$(f_j - f_i) < -1 \rightarrow c_e = C_{before}$$

The cost of each edge, c_e must be multiplied by the *likelihood* of that edge being exercised, p_e , which is also captured by the field-access graph, in the form of edge weights. Combining these gives us a total estimated cost for any particular field layout:

$$C = \sum_{e \in E} c_e \cdot p_e \quad (5)$$

This is the cost that our ILP attempts to minimize, subject to the hard constraints 1 and 2.

3.6.4 Greedy layout ordering. Since there is a tradeoff in finding optimal layout orders and compile times when using a solver based optimization. We implemented a greedy algorithm that traverses the access graph for a data constructor in a greedy fashion. It starts from the root node of the access graph which is the first field in the layout of the data constructor and greedily (based on the edge weight) visits the next field. We fix the edge order for a control flow edge as the original order and do not look at field attributes. However, after the greedy algorithm picks a layout we match

the let expressions to the layout order to make sure the code matches the layout order. A greedy algorithm is potentially sub-optimal when it comes to finding the best performing layout, however, the compile time is fast.

3.7 Finding a global layout

A data constructor can be used across multiple functions, therefore, we need to find a layout order that is optimal globally. To do so, we take constraints for each function and data constructor pair and combine them uniformly, that is, a uniform weight for each function. Dynamic profiling techniques can be used to get more precise weights, but, we leave such efforts for future work. We then feed the combined constraints to the solver to get a globally optimal layout for that data constructor. The global optimization finds a globally optimal layout for all data constructors in the program. Once the new global layout is chosen for a data constructor, we re-write the entire program to adhere to the new layout and subsequently typecheck. In the evaluation, we use the global optimization. However, we only show the data constructor that constitutes the major part of the program.

4 IMPLEMENTATION

We implement MARMOSSET in the open-source Gibbon compiler⁵. Gibbon is a whole-program micropass compiler that compiles a polymorphic, higher-order subset of (strict) Haskell. The Gibbon front-end uses standard whole-program compilation and monomorphization techniques [Chlipala 2015] to lower input programs into a first-order, monomorphic IR (λ_M). Gibbon performs location inference on this IR to convert it into a LoCal program, which has regions and locations, essentially, buffers and pointer arithmetic. Then a big middle section of the compiler is a series of LoCal- $\rightarrow$ LoCal compiler passes that perform various transformations. Finally, it generates C code. Our extension operates towards the front-end of the compiler, on λ_M . We closely follow the design described in Section 3 to construct the control-flow graph and field-access graph, and use the standard Haskell graph library⁶ in our implementation.

To solve the constraints, we use IBM's DOCPLEX (Decision Optimization CPLEX), because its API allows high level modelling such as logical expressions like implications, negations, logical AND etc. with relatively low overhead. Unfortunately, there isn't a readily available Haskell library that can interface with DOCPLEX. Thus, we use it via its library bindings available for Python. Specifically, we generate a Python program that feeds the constraints to DOCPLEX and outputs an optimum field ordering to the standard output, which MARMOSSET reads and parses, and then reorders the fields accordingly.

5 EVALUATION

We evaluate MARMOSSET on three applications. First is a pair of microbenchmarks: a list length function (Section 5.2.1) and a logical expression evaluator (Section 5.2.2). Second is a small library of operations with binary trees (Section 5.3). Third is a blog management software based on the BLogList example from the Sections 2–3 (Section 5.4). Besides the run times, we take a closer look at how MARMOSSET affects compile times (Section 5.6) and cache behavior at run time (Section 5.5).

We detail the impact of various datatype layouts on the performance. As the baseline, we use GIBBON, the most closely related prior work. We also compare to GHC where appropriate⁷. For

⁵<https://github.com/iu-parfunc/gibbon/>

⁶<https://hackage.haskell.org/package/containers>

⁷The reader should keep in mind that GHC solves a much harder task of compiling the giant Haskell language with dozens of extensions. Our goal in the brief account of GHC's performance relative to MARMOSSET is to show the potential of using a dense representation of recursive datatypes with automatic layout management, a technique that could be incorporated in part into GHC.

Table 2. List length run times (9 runs): Mean, Median and 95% confidence interval (ub, lb) in milliseconds (ms) for different layouts when compiled with GIBBON. The last two columns show the runtime for the layout chosen by M_{Greedy} and M_{Solver} . The numbers in **Blue** correspond to the lowest running time and the numbers in **Red** correspond to the highest running time.

	GIBBON		MARMOSET	
	List	List'	M_{Greedy}	M_{Solver}
Mean	59.958ms	1.027ms	1.172ms	1.099ms
Median	59.872ms	1.021ms	1.098ms	1.101ms
ub	60.094ms	1.094ms	1.303ms	1.103ms
lb	59.823ms	0.992ms	1.041ms	1.095ms

each benchmark, we run 9 iterations and report the run-time mean, median and the 95% confidence interval.

5.1 Experimental Setup

We run our benchmarks on a server-class machine with 64 CPUs, each with two threads. The CPU model is AMD Ryzen Threadripper 3990X with 2.2 GHz clock speed. The L1 cache size is 32 KB, L2 cache size is 512 KB and L3 cache size is 16 MB. We use GIBBON's default C backend and call GCC 10.2.0 with `-O3` to generate binaries.

5.2 Micro Benchmarks

We use two microbenchmarks to explore specific performance penalties imposed by a sub-optimal data layout.

5.2.1 Computing the length of a linked-list. This benchmark demonstrates the cost of de-referencing memory addresses which are not present in the cache. It uses the linked list datatype:

```
data List = Nil | Cons Content List
```

If each element of the list is constructed using `Cons`, the traversal has to de-reference a pointer—to *jump over* the content—each time to access the tail of the list. This is an expensive operation, especially if the target memory addresses is not present in the cache. On the contrary, if the `Content` and `List` fields were swapped, then to compute the length, the program only has to traverse n bytes for a list of length n —one byte per `Cons` tag—which is extremely efficient. Essentially, MARMOSET transforms program to use the following datatype, while preserving its behavior:

```
data List' = Nil | Cons List' Content
```

In our experiment, the linked list is made of 3M elements and each element contains some content (a `Pandoc Inline` data type) that occupies roughly 5KB. As seen in table 2, the performance of the list constructed using the original `Cons` is $\sim 54\times$ worse than the performance with the MARMOSET-optimized, flipped layout. Not only does the `Cons`-list have poor data locality and cache behavior, but it also has to execute more instructions to de-reference the pointer. Both M_{Greedy} and M_{Solver} choose the flipped layout `List'`.

5.2.2 Short-circuiting logical And/Or operators. These microbenchmarks operate over a synthetically generated, balanced syntax-tree with a height of 30. The intermediate nodes can be one of `Not`, `Or`, or `And`, selected at random, and the leaves consist of booleans wrapped in a `Val`. The evaluator function evaluates the expression tree and performs short circuiting for expressions

Table 3. Logical expression evaluator run times. Legend: l - left subtree, r - right subtree of the decision tree.

	GIBBON		MARMOSSET	
	lr	rl	M_{Greedy}	M_{Solver}
Mean	3.74s	6.27s	3.51s	3.49s
Median	3.76s	6.27s	3.51s	3.49s
ub	3.77s	6.28s	3.51s	3.50s
lb	3.70s	6.27s	3.51s	3.49s

like `and` and `or` in the expression tree. We measure the performance of this evaluator function by changing the order of the left and right subtrees. The syntax-tree datatype is defined as follows:

```
data Exp = Val Bool | Not Exp | Or Exp Exp | And Exp Exp
```

Since the short circuiting evaluates from left to right order of the `Exp`, changing the order of the left and right subtrees would affect the performance of the traversal. As it can be seen in table 3, the layout where the left subtree is serialized before the right subtree results in better performance compared to the tree where the the right subtree is serialized before the left one. This is as expected since in the latter case, the traversal has to jump over the right subtree serialized before the left one in order to evaluate it first and then depending on the result of the left subtree possibly jump back to evaluate the right subtree. This results in poor spatial locality and hence worse performance. M_{Greedy} and M_{Solver} are able to identify the layout transformations that would give the best performance and matches the case where the left subtree is serialized before the right subtree.

5.3 Binary Tree benchmarks

We evaluate MARMOSSET on a few binary tree benchmarks. These include adding one to all values stored in the internal nodes of the tree, exponentiation on integers stored in internal nodes, copying a tree and getting the right-most leaf value in the tree. For the first three benchmarks, the tree representation we use is:

```
data Tree = Leaf | Node Int Tree Tree
```

For right-most, the tree representation we use is:

```
data Tree = Leaf Int | Node Tree Tree
```

5.3.1 Add one tree. This benchmark takes a full binary tree and increments the values stored in the internal nodes of the tree. We show the performance of an aligned preorder, inorder and postorder traversal in addition to a misaligned preorder and postorder traversal of the tree. Aligned traversals are ones where the data representation exactly matches the traversal order, for instance, a preorder traversal on a preorder representation of the tree. A misaligned traversal order is where the access patterns of the traversal don't match the data layout of the tree. For instance, a postorder traversal on a tree serialized in preorder. Table 4 shows the performance numbers. We show also the performance of M_{Greedy} and M_{Solver} . M_{Solver} picks the aligned postorder traversal order which is best performing. It makes the recursive calls to the left and right children of the tree first and increments that values stored in the internal nodes once the recursive calls return. The tree representation is also changed to a postorder representation with the `Int` placed after the left and right children of the tree. The tree depth was set to 27. At this input size, the `MISALIGNEDPost` traversal failed due to memory errors. We see significant slowdowns with `MISALIGNEDPre`, ~35× slower, because of the skewed access patterns of the traversal.

Table 4. Binary Tree benchmarks runtime statistics. MISALIGNED_{Pre}– A post order traversal on a pre order layout of the tree. MISALIGNED_{Post}– A pre order traversal on a post order layout of the tree. ALIGNED_{Pre}– A pre order traversal on a pre order layout of the tree. ALIGNED_{In}– An in order traversal on an in order layout of the tree. ALIGNED_{Post}– A post order traversal on a post order layout of the tree.

	GIBBON					MARMOSSET	
	MISALIGNED _{Pre}	MISALIGNED _{Post}	ALIGNED _{Pre}	ALIGNED _{In}	ALIGNED _{Post}	M _{Greedy}	M _{Solver}
Add One Tree							
Mean	4.551e1s	memory error	1.323s	1.324s	1.314s	1.315s	1.309s
Median	4.551e1s	memory error	1.293s	1.293s	1.284s	1.285s	1.278s
ub	4.566e1s	memory error	1.391s	1.393s	1.385s	1.385s	1.380s
lb	4.535e1s	memory error	1.254s	1.254s	1.244s	1.245s	1.238s
Exponentiation Tree							
Mean	4.552e1s	memory error	1.341s	1.339s	1.327s	1.336s	1.316s
Median	4.557e1s	memory error	1.310s	1.31s	1.296s	1.305s	1.286s
ub	4.570e1s	memory error	1.411s	1.400s	1.41s	1.406s	1.386s
lb	4.534e1s	memory error	1.270s	1.268s	1.254s	1.265s	1.246s
Copy Tree							
Mean	4.552e1s	memory error	1.318s	1.320s	1.311s	1.317s	1.299s
Median	4.553e1s	memory error	1.288s	1.289s	1.280s	1.286s	1.268s
ub	4.567e1s	memory error	1.388s	1.389s	1.384s	1.389s	1.369s
lb	4.537e1s	memory error	1.248s	1.250s	1.238s	1.246s	1.228s

5.3.2 *Exponentiation Tree*. This traversal does exponentiation on the values stored in the internal nodes of the tree. It is more computationally intensive than incrementing the value. We raise the `Int` to a power of 10 on a tree of depth 27. Table 4 shows the performance of the different layout and traversal orders. The solver picks the ALIGNED_{Post} representation which is the best performing.

5.3.3 *Copy Tree*. Copy-tree takes a full binary tree and makes a fresh copy of the tree in a new memory location. We use a tree of depth 27 in our evaluation. Table 4 shows the performance of different layout and traversal orders. We see that ALIGNED_{Post} traversal performs the best. Indeed, M_{Solver} picks the ALIGNED_{Post} traversal.

5.3.4 *Righmost tree*. This traversal does recursion on the right child of the tree and returns the `Int` value stored in the right-most leaf of the tree. The running time of the benchmark is significantly lower than the time it takes to build the tree. It is *logarithmic* in the number of nodes in the tree. We used the largest tree size (depth=32) we could on the machine we ran the benchmarks on. As shown in table 5, we see that the representation with the right child serialized before the left child of the tree performs best as expected. The traversal does not have to jump over the left subtree to access the right subtree. M_{Solver} picks the layout with the right subtree serialized before the left subtree.

5.4 Blog Software Case Study

The Blog software case study serves as an example of a realistic benchmark, representing a sample of components from a blog management web-service. The main data structure is a linked-list of blogs where each blog contains various fields such as header, id, author information, content, hashtags and date. The fields are a mix of recursive and non-recursive datatypes. For instance,

Table 5. Rightmost tree runtime statistics. Legend: l - left subtree, r - right subtree of the mono tree.

	GIBBON		MARMOSET	
	lr	rl	M_{Greedy}	M_{Solver}
Mean	1.922e - 7s	1.878e - 7s	1.544e - 7s	1.556e - 7s
Median	1.600e - 7s	1.400e - 7s	1.400e - 7s	1.400e - 7s
ub	2.623e - 7s	2.064e - 7s	2.088e - 7s	1.944e - 7s
lb	1.221e - 7s	1.136e - 7s	1.179e - 7s	1.167e - 7s

Table 6. Runtime statistics for different blog traversals for a subset permutation of different layouts when compiled with GIBBON. Legend: h - Header, t - HashTags, b - Blogs, i - TagID, c - Content, a - Author, d - Date.

	GIBBON							MARMOSET	
	hiadctb	ctbhiad	tbchiad	tcbhiad	btchiad	bchiadt	cbiadt	M_{Greedy}	M_{Solver}
Filter Blogs									
Mean	0.246s	0.236s	0.089s	0.268s	0.293s	0.297s	0.221s	0.082s	0.082s
Median	0.256s	0.244s	0.101s	0.272s	0.300s	0.304s	0.232s	0.090s	0.087s
ub	0.254s	0.243s	0.101s	0.276s	0.300s	0.305s	0.237s	0.091s	0.091s
lb	0.239s	0.228s	0.078s	0.261s	0.285s	0.289s	0.204s	0.073s	0.073s
Emph Content									
Mean	0.740s	0.718s	1.280s	0.724s	1.261s	1.264s	0.576s	0.611s	0.656s
Median	0.798s	0.770s	1.280s	0.777s	1.262s	1.265s	0.654s	0.693s	0.640s
ub	0.775s	0.760s	1.283s	0.760s	1.268s	1.269s	0.624s	0.704s	0.660s
lb	0.705s	0.676s	1.276s	0.687s	1.253s	1.258s	0.527s	0.562s	0.620s
Tag Search									
Mean	2.223s	2.235s	2.753s	2.000s	2.758s	2.787s	2.111s	2.050s	1.791s
Median	2.418s	2.435s	2.754s	2.257s	2.756s	2.789s	2.356s	2.300s	1.723s
ub	2.339s	2.355s	2.759s	2.149s	2.762s	2.790s	2.258s	2.198s	1.947s
lb	2.107s	2.115s	2.748s	1.851s	2.754s	2.783s	1.964s	1.903s	1.636s

`Content` is a recursive type (a *Pandoc* Block type), but `Author` is just a single string wrapped in a data constructor. Below we show one possible permutation of fields contained in each blog.

```
data Blogs = Empty | HIADCTB Header Id Author Date Content HashTags Blogs
```

We evaluate MARMOSET's performance using three different traversals over a list of blogs. Overall, these traversals only use three fields: `Content` and `Tags`, and the tail of the linked-list, `Blogs`. The fields used by each individual traversal are referred as *active fields* and the rest are referred as *passive fields*, and we specify these per-traversal in the following. In Table 6 we report the performance of all six layouts obtained by permuting the order of these three fields, and two additional layouts (Columns 1 and 2). The column names are indicative of the order of fields used. For example, the column `hiadctb` reports numbers for a layout where the fields are ordered as: `Header`, `Id`, `Author`, `Date`, `Content`, `HashTags`, and `Blogs` (the tail of the list). We use GIBBON to gather these run times. The second to last and the last column shows the run times of these traversals using MARMOSET's greedy and solver based optimization. The layout it picks is tailored to each individual traversal so we specify it per-traversal next.

5.4.1 FilterBlogs. This traversal filters the list of blogs and only retains those which contain the given hash-tag in their `HashTags` field. Since `Blogs`, `Content`, and `HashTags` are recursive fields, changing their layout should represent greater differences in performance. The *active fields* for this traversal are `HashTags` and `Blogs`. Theoretically, the performance of this traversal is optimized when the `HashTags` field is serialized before `Blogs` on account of the first access to `HashTags` in the traversal. This is confirmed in practice with the layout *tbchiad* being the fastest. The access to `HashTags` comes first since its serialized first in memory and once `HashTags` is fully traversed, the traversal can access `blogs` in an optimal manner without unnecessary movement in the packed buffer.

Indeed, the layout chosen by MARMOSET puts `HashTags` first followed by `Blogs`. The order of the rest of the fields remains unchanged compared to the source program, but this has no effect on performance since they are *passive fields*. As shown in Table 6, the performance of the layout chosen by MARMOSET is similar to the layout *tbchiad* when compiled with GIBBON.

5.4.2 EmphContent. This traversal takes a keyword and a list of blogs as input. It then searches the content of each blog for the keyword and emphasizes all its occurrences if present. If the keyword isn't present, the content remains unchanged. The active fields in this traversal are `Content` and `Blogs`. Therefore, based on the access pattern (`Content` accessed before `Blogs`), the layout with the best performance should place `Content` first followed by `Blogs`. Indeed, the layout with the best performance when compiled with GIBBON is *cbiadht*. The layout that M_{Solver} chooses prioritizes the placement of `Blogs` before `Content`. It subsequently changes the traversal to recurse on the blogs first and then emphasize content. The other passive fields are all placed afterwards. As shown in table 6, the performance of the layout chosen by M_{Greedy} and M_{Solver} are similar to *cbiadht*.

5.4.3 TagSearch. This traversal takes a keyword and a list of blogs as input. It looks for the presence of a keyword in the `HashTags` field. If it is present, it emphasizes this keyword in the `Content`, and leaves it unchanged otherwise. The layout that gives the best performance when compiled with GIBBON is *tcchiad*. This is as expected based on the access patterns of the traversal which access `HashTags` followed by `Content` followed by `Blogs`.

The layout MARMOSET's solver version chooses places `HashTags`, followed by `Blogs`, followed by `Content` and changes the traversal to recurse on `Blogs` first and later emphasize `Content` in the *then* branch. MARMOSET's performance is better and shows the effectiveness of the optimization.

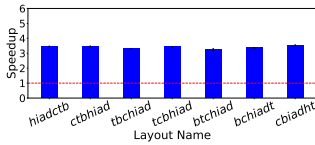
5.4.4 Globally optimizing multiple functions. We use MARMOSET to globally optimize the three blog traversals we discussed above such that we pick one layout for all traversals that minimizes the overall runtime. Table 7 shows the runtime for a layout we compiled using GIBBON (*hiadctb*). It shows the runtime for the greedy optimization (M_{Greedy}) and the solver optimization (M_{Solver}). We see that M_{Greedy} and M_{Solver} do a good job in reducing the traversal time globally. All the three traversals are run in a pipelined fashion sequentially. Although, M_{Solver} does worse with *TagSearch* when run in a pipelined manner, it is actually better performing with M_{Solver} when run alone as seen in table 6. Since M_{Solver} changes more than one data constructor based on the *inlineable* attribute that M_{Greedy} does not, we think that it performs worse in a pipeline due to a cache effect that we haven't accounted for. However, as mentioned previously, dynamic profiling can help us in such cases to come up with more optimal layouts for each data constructor.

5.4.5 Comparison of MARMOSET against GIBBON and GHC. We compare MARMOSET's performance to GIBBON, and also GHC, which is especially optimized to run functional programs.

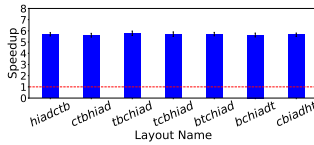
Figure 9 shows the the speedup of MARMOSET over GHC when evaluated strictly for an apples to apples comparison. We pass the optimization option *-O2* to GHC while compiling. As shown, the performance of MARMOSET is better than GHC by significant margins for all the layouts and

Table 7. Runtime for different blog traversals in a pipeline when we use MARMOSET to globally optimize the data layout. The input parameters are different from the single function case.

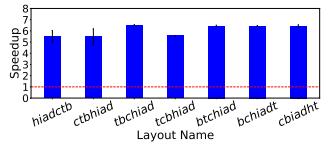
	GIBBON	M_{Greedy}	M_{Solver}
Filter Blogs			
Mean	1.680s	0.107s	0.106s
Median	1.681s	0.102s	0.101s
ub	1.684s	0.115s	0.116s
lb	1.676s	0.099s	0.095s
Emph Content			
Mean	1.928s	1.433s	1.374s
Median	1.925s	1.391s	1.318s
ub	1.934s	1.531s	1.502s
lb	1.921s	1.336s	1.245s
Tag Search			
Mean	2.560s	1.885s	2.391s
Median	2.561s	1.843s	2.347s
ub	2.565s	1.983s	2.290s
lb	2.555s	1.787s	2.492s



(a) Filter Blogs Traversal.



(b) Content Search Traversal.



(c) Tag Search Traversal.

Fig. 9. Performance comparison of M_{Solver} with GHC.

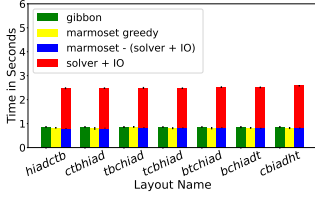
traversals. Since the types in GHC are *boxed*, such a behavior is expected because it adds more instructions (pointer de-referencing) and results in worse spatial locality.

5.5 Cache behavior

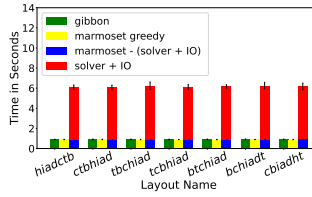
The results from earlier sections demonstrate that MARMOSET's layout choices do, indeed, improve performance. This section investigates *why* performance improves. The basic premise of MARMOSET's approach to layout optimization is to concentrate on minimizing how often a traversal needs to backtrack or skip ahead while processing a buffer. By minimizing this jumping around, we expect to see improvements from two possible sources. First, we expect to see an improvement in instruction counts, as an optimized layout should do less pointer chasing. Second, we expect to see an improvement in L2 and L3 cache utilization: both fewer misses (due to improved spatial locality and prefetching) and fewer accesses (due to improved locality in higher level caches).

Table 8. PAPI performance counter statistics (average of 9 runs) for different blog traversals.

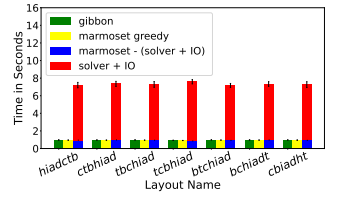
	GIBBON							MARMOSET	
	hiadctb	ctbhiad	tbchiad	tcbhiad	btchiad	bchiadt	cbiadht	M_{Greedy}	M_{Solver}
Filter Blogs									
Ins	5.83e8	5.82e8	5.83e8	5.81e8	5.79e8	5.78e8	5.86e8	5.83e8	5.83e8
Cycles	9.89e8	9.60e8	2.85e8	1.05e9	1.20e9	1.25e9	8.78e8	2.79e8	2.89e8
L2 DCM	1.12e7	1.15e7	9.38e5	1.33e7	1.29e7	1.31e7	7.79e6	8.90e5	8.75e5
Content Search									
Ins	5.85e9	5.83e9	6.78e9	5.84e9	6.78e9	6.78e9	5.84e9	5.84e9	5.84e9
Cycles	2.89e9	2.74e9	4.27e9	2.84e9	4.34e9	4.29e9	2.06e9	2.06e9	2.81e9
L2 DCM	1.28e7	1.08e7	2.03e7	1.33e7	2.05e7	2.1e7	7.73e6	7.66e6	1.07e7
Tag Search									
Ins	2.25e10	2.25e10	2.30e10	2.25e10	2.30e10	2.30e10	2.25e10	2.25e10	2.25e10
Cycles	8.59e9	8.59e9	9.61e9	7.29e9	9.61e9	9.75e9	7.88e9	7.61e9	7.57e9
L2 DCM	2.02e7	2.06e7	4.00e7	1.06e7	2.86e7	2.65e7	1.64e7	1.29e7	1.28e7



(a) Compile times for the Filter blogs traversal.



(b) Compile times for the Content Search traversal.



(c) Compile times for the Tag Search traversal.

Fig. 10. Average compile times (9 runs) in seconds for different layouts and traversal combinations when compiled with GIBBON and when optimized for layout by M_{Greedy} and M_{Solver} . The bar in Blue is the compile time in seconds for M_{Solver} after subtracting the time for solver IO and the bar in Red is the time taken to do the IO call to the DOCPLEX solver. The total compile time for M_{Solver} includes the time for doing the CFG analysis, generating the access graph, the solver time, reordering the data type and repairing the code. The majority of which is added by the solver IO.

To substantiate this hypothesis, we collected performance counter measurements for various layouts. The results are shown in Table 8.⁸ We see that our main hypothesis is borne out. The optimal layout indeed has far fewer L2 data cache misses: the better layout promotes better locality. Interestingly, we do not observe a similar effect for instruction count. While different layouts do differ somewhat in instructions, the difference is slight. We suspect this is partially due to GIBBON's current implementation, that often dereferences pointers even if a simple access in the buffer would suffice.

5.6 Compile times of the different layouts when compiled with GIBBON and MARMOSET

We show the compile times in Figure 10 for different layout and traversal combinations when compiled with GIBBON and when optimized with MARMOSET respectively. The compile times for MARMOSET include the time to generate the control flow graph, the field access graph, the solver time, the time to re-order the data type and finally repairing the code so that is typechecks. The solver times are in the order of the number of fields in a data constructor and not the program size. Hence, the solver adds relatively low overhead. Since the compiler does an IO call to the python solver, there is room for improvement in the future to lower these times. For instance, we could directly perform *FFI* calls to the CPLEX solver by lowering the constraints to C code in MARMOSET. This would be faster and safer than the current implementation. In addition, during the global optimization, we call the solver on each data constructor as of the moment, we could further optimize this by sending constraints for all data constructors at once and doing just one solver call.

6 DISCUSSION

Throughout the case studies, we see that MARMOSET can optimize the layout of datatypes and provide performance gains. We believe that these gains can be further improved by exploring ideas briefly discussed in this section.

MARMOSET allows the user to provide optional manual constraints on the layout as either relative or absolute constraints through *ANN* annotation pragmas. The relative constraint allows the user to specify if a field A comes immediately after field B. The absolute constraint specifies an exact index in the layout for a field. Such pragmas may be beneficial if the user requires a specific configuration over data types or has information about performance bottlenecks.

Although the performance optimization is currently statically driven, there are many avenues for future improvement. For instance, we can get better edges weights for the access graphs using dynamic profiling techniques. The profiling can be quite detailed, for instance, which branch in a function is more likely, which function takes the most time overall in a global setting (the optimization would bias the layout towards that function), how does a particular global layout affect the performance in case of a pipeline of functions.

We could also look at a scenario where we optimize each function locally and use “shim” functions that copy one layout to another (the one required by the next function in the pipeline). Although the cost of copying may be high, it warrants further investigation. Areas of improvement purely on the implementation side include optimizing whether MARMOSET dereferences a pointer to get to a field or uses the end-witness information as mentioned in section 2. Lesser pointer dereferencing can lower instruction counts and impact performance positively. We would also like to optimize the solver times as mentioned in 5.6.

We envision that the *structure of arrays* effect that we discovered may help with optimizations such as vectorization, where, the performance can benefit significantly if the same datatype is close together in memory. Regardless, through the case studies, we see that MARMOSET shows promise in optimizing the layout of datatypes and may open up the optimization space for other complex optimizations such as vectorization.

7 RELATED WORK

7.1 Cache-conscious data

Using *generational garbage collection to implement cache-conscious data placement in object-oriented languages* [Chilimbi and Larus 1998]. Their approach builds on an object-oriented language

⁸Since PAPI does not completely support latest AMD processors yet, we were unable to obtain L3 cache misses, only the L2 DCM counter was available.

with a generational garbage collector, which they extend with a heuristic for copying objects to the TO space. Their heuristic uses a special-purpose graph data structure, the *object affinity graph*, to identify when groups of objects are accessed by the program close together in time. When a given group of objects have high affinity in the object affinity graph, the collector is more likely to place them close together in the TO space. As such, a goal of their work and ours is to achieve higher data-access locality by carefully grouping together objects in the heap. However, a key difference from our work is that their approach bases its placement decisions on an object-affinity graph that is generated from profiling data, which is typically collected online by some compiler-inserted instrumentation. The placement decisions made by our approach are based on data collected by static analysis of the program. Such an approach has the advantage of not depending on the output of dynamic profiling, and therefore avoids the implementation challenges of dynamic profiling. A disadvantage of not using dynamic profiling is that the approach cannot adapt to changing access patterns that are highly input specific. We leave open for future work the possibility of getting the best of both approaches.

Cache-Conscious Structure Definition [Chilimbi et al. 1999a]. This work introduces the idea of hot/cold splitting of a data structure, where elements are categorized as being “hot” if accessed frequently and “cold” if accessed infrequently. This information is obtained by profiling the program. Cold fields are placed into a new object via an indirection and hot fields remain unchanged. In their approach, at runtime, there is a cache-conscious garbage collector [Chilimbi and Larus 1998] that co-locates the modified object instances. This paper also suggests placing fields with high temporal affinity within the same cache block. For this they recommend bbcache, a field recommender for a data structure. bbcache forms a field affinity graph which combines static information about the source location of structure field accesses with dynamic information about the temporal ordering of accesses and their access frequency.

Cache-Conscious Structure Layout [Chilimbi et al. 1999b]. This paper proposes two techniques to solve the problem of poor reference locality in user programs.

- **ccmorph**: This works on tree-like data structures, and it relies on the programmer making a calculated guess about the safety of the operation on the tree-like data structure. It performs two major optimizations: clustering and coloring. Clustering takes a tree-like data structure and attempts to pack likely to be accessed elements in the structure within the same cache block. There are various ways to pack a subtree, including clustering k nodes in a subtree together, depth first clustering, etc. Coloring attempts to map simultaneously accessed data elements to non-conflicting regions of the cache.
- **ccmalloc**: This is a memory allocator similar to malloc which takes an additional parameter that points to an existing data structure element which is likely accessed simultaneously. This requires programmer knowledge and effort in recognizing and then modifying the code with such a data element. ccmalloc tries to allocate the new data element as close to the existing data as possible, with the initial attempt being to allocate in the same cache block. It tries to put likely accessed elements on the same page in an attempt to improve TLB performance.

SHAPES [Franco et al. 2017, 2019]. The idea of this work is that the layout of a data structure should be defined once at the point of initialization, and all further code that interacts with the structure should be “layout agnostic”. Ideally, this means that performance improvements involving layout changes can be made without requiring changes to program logic. To achieve this, classes are extended to support different layouts, and types carry layout information—code that operates on objects may be polymorphic over the layout details.

7.2 Data layout description and binary formats

Dargent [Chen et al. 2023]. This work proposes a data layout description framework that allows programmers to specify the data layout of an algebraic data type. It is built on top of the Cogent language [O'Connor et al. 2016], which is a first order polymorphic functional programming language. Dargent targets C code and provides proofs of formal correctness of the compiled C code with respect to the layout descriptions. Rather than having a compiler attempting to determine an efficient layout, their focus is on allowing the programmer to specify a particular layout they want and have confidence in the resulting C code.

Additionally, there has been significant prior work on the generation of verified, efficient code for interacting with binary data formats (especially parsing and validating). For example, Ever-Parse [Ramanananth et al. 2019] is a framework for generating verified parsers and formatters for binary data formats, and it has been used to formally verify zero-copy parsers for authenticated message formats. With Narcissus [Delaware et al. 2019], encoders and decoders for binary formats could be verified and extracted, allowing researchers to certify the packet processing for a full internet protocol stack. Other work [van Geest and Swierstra 2017] has also explored the automatic generation of verified parsers and pretty printers given a specification of a binary data format, as well as the formal verification of a compiler for a subset of the Protocol Buffer serialization format [Ye and Delaware 2019].

Earlier work [Back 2002] demonstrated how a domain-specific language for describing binary data formats could be useful for generating validators and for easier scripting and manipulation of the data from a high-level language like Java. In other earlier related work, Packet Types [McCann and Chandra 2000] used types for programming with network protocol messages, with language-level support for features commonly found in protocol formats like variable-sized and optional fields.

Data Representation Synthesis [Hawkins et al. 2012] introduces RELC, a framework for synthesizing low-level C++ code from a high level relational representation of the code. The user describes and writes code that represents data at a high level as relations. Together with a decomposition of the data that outlines how that data should be represented in memory, RELC, synthesizes correct and efficient low level code from those descriptions.

7.3 Memory layouts

Floorplan [Cronburg and Guyer 2019]. This work proposes a declarative language for specifying high-level memory layouts, implemented as a compiler which generates Rust code. The language has forms for specifying sizes, alignments, and other features of chunks of memory in the heap, with the idea that any correct state of the heap can be derived from the Floorplan specification. It was successfully used to eliminate 55 out of 63 unsafe lines of code (all of the unsafe code relating to memory safety) of the immix garbage collector.

Early work on specifications of memory layouts was explored in various studies of PADS [Fisher and Gruber 2005; Fisher and Walker 2011; Mandelbaum et al. 2007], a language for describing ad hoc data-file formats.

Automatic Pool Allocation for Disjoint Data Structures [Lattner and Adve 2002]. This paper introduces a technique for improving the memory layout of the heap of a given C program. Their approach is to use the results of a custom static analysis to enable pool allocation of objects allocated on the heap. Such automatic pool allocation bears some resemblance to our approach, where we use region-based allocation in tandem with region inference, and thanks to static analysis can group fields of a given struct into the same pool, thereby improving locality in certain circumstances.

8 CONCLUSIONS

This paper introduced MARMOSSET, which builds on Gibbon to generate efficient orders for Algebraic Data Types. We observed that a fairly straightforward control flow and data flow analysis allows MARMOSSET to identify opportunities to place fields of a data constructor near each other in memory to promote efficient consecutive access to those fields. Because a given function might use many fields in many different ways, MARMOSSET adopts an approach of formulating the data layout problem as an ILP, with a cost model that assigns an abstract cost to a chosen layout. This ILP can be further constrained with user-provided layout guidelines. Armed with problem formulation, an off-the-shelf ILP solver allows MARMOSSET to generate minimal-(abstract)-cost layouts for algebraic data types. MARMOSSET then uses these layouts to synthesize a new ADT and prior work's Gibbon compiler toolchain to lower the code into an efficient program that operates over packed data types with minimal pointer chasing.

We found, across a number of benchmarks, that MARMOSSET is able to effectively and consistently find the optimal data layout for a given combination of traversal function and ADT. We showed, experimentally, that MARMOSSET-optimized layouts significantly outperform not only Gibbon's default layouts, but also GHC's standard compiler.

REFERENCES

- Godmar Back. 2002. DataScript - A Specification and Scripting Language for Binary Data. In *Proceedings of the 1st ACM SIGPLAN/SIGSOFT Conference on Generative Programming and Component Engineering (GPCE '02)*. Springer-Verlag, Berlin, Heidelberg, 66–77.
- Zilin Chen, Ambroise Lafont, Liam O'Connor, Gabriele Keller, Craig McLaughlin, Vincent Jackson, and Christine Rizkallah. 2023. Dargent: A Silver Bullet for Verified Data Layout Refinement. *Proc. ACM Program. Lang.* 7, POPL, Article 47 (jan 2023), 27 pages. <https://doi.org/10.1145/3571240>
- Trishul M. Chilibi, Bob Davidson, and James R. Larus. 1999a. Cache-Conscious Structure Definition. In *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation* (Atlanta, Georgia, USA) (PLDI '99). Association for Computing Machinery, New York, NY, USA, 13–24. <https://doi.org/10.1145/301618.301635>
- Trishul M. Chilibi, Mark D. Hill, and James R. Larus. 1999b. Cache-Conscious Structure Layout. In *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation* (Atlanta, Georgia, USA) (PLDI '99). Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/301618.301633>
- Trishul M. Chilibi and James R. Larus. 1998. Using Generational Garbage Collection to Implement Cache-Conscious Data Placement. In *Proceedings of the 1st International Symposium on Memory Management* (Vancouver, British Columbia, Canada) (ISMM '98). Association for Computing Machinery, New York, NY, USA, 37–48. <https://doi.org/10.1145/286860.286865>
- Adam Chlipala. 2015. An Optimizing Compiler for a Purely Functional Web-application Language. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming* (Vancouver, BC, Canada) (ICFP 2015). ACM, New York, NY, USA, 10–21. <https://doi.org/10.1145/2784731.2784741>
- Karl Cronburg and Samuel Z. Guyer. 2019. Floorplan: Spatial Layout in Memory Management Systems. In *Proceedings of the 18th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences* (Athens, Greece) (GPCE 2019). Association for Computing Machinery, New York, NY, USA, 81–93. <https://doi.org/10.1145/3357765.3359519>
- Benjamin Delaware, Sorawit Suriyakarn, Clément Pit-Claudel, Qianchuan Ye, and Adam Chlipala. 2019. Narcissus: Correct-by-Construction Derivation of Decoders and Encoders from Binary Formats. *Proc. ACM Program. Lang.* 3, ICFP, Article 82 (jul 2019), 29 pages. <https://doi.org/10.1145/3341686>
- Kathleen Fisher and Robert Gruber. 2005. PADS: A Domain-Specific Language for Processing Ad Hoc Data. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation* (Chicago, IL, USA) (PLDI '05). Association for Computing Machinery, New York, NY, USA, 295–304. <https://doi.org/10.1145/1065010.1065046>
- Kathleen Fisher and David Walker. 2011. The PADS Project: An Overview. In *Proceedings of the 14th International Conference on Database Theory* (Uppsala, Sweden) (ICDT '11). Association for Computing Machinery, New York, NY, USA, 11–17. <https://doi.org/10.1145/1938551.1938556>
- Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. 1993. The Essence of Compiling with Continuations. In *Proceedings of the ACM SIGPLAN 1993 Conference on Programming Language Design and Implementation* (Albuquerque, New Mexico, USA) (PLDI '93). Association for Computing Machinery, New York, NY, USA, 237–247. <https://doi.org/10.1145/155090.155113>

- Juliana Franco, Martin Hagelin, Tobias Wrigstad, Sophia Drossopoulou, and Susan Eisenbach. 2017. You Can Have It All: Abstraction and Good Cache Performance. In *Proceedings of the 2017 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Vancouver, BC, Canada) (*Onward! 2017*). Association for Computing Machinery, New York, NY, USA, 148–167. <https://doi.org/10.1145/3133850.3133861>
- Juliana Franco, Alexandros Tasos, Sophia Drossopoulou, Tobias Wrigstad, and Susan Eisenbach. 2019. Safely Abstracting Memory Layouts. <https://doi.org/10.48550/ARXIV.1901.08006>
- Peter Hawkins, Alex Aiken, Kathleen Fisher, Martin Rinard, and Mooly Sagiv. 2012. Concurrent Data Representation Synthesis. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation* (Beijing, China) (*PLDI '12*). Association for Computing Machinery, New York, NY, USA, 417–428. <https://doi.org/10.1145/2254064.2254114>
- Chris Lattner and Vikram Adve. 2002. Automatic Pool Allocation for Disjoint Data Structures. In *Proceedings of the 2002 Workshop on Memory System Performance* (Berlin, Germany) (*MSP '02*). Association for Computing Machinery, New York, NY, USA, 13–24. <https://doi.org/10.1145/773146.773041>
- Yitzhak Mandelbaum, Kathleen Fisher, David Walker, Mary Fernandez, and Artem Gleyzer. 2007. PADS/ML: A Functional Data Description Language. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Nice, France) (*POPL '07*). Association for Computing Machinery, New York, NY, USA, 77–83. <https://doi.org/10.1145/1190216.1190231>
- Peter J. McCann and Satish Chandra. 2000. Packet Types: Abstract Specification of Network Protocol Messages. In *Proceedings of the Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication* (Stockholm, Sweden) (*SIGCOMM '00*). Association for Computing Machinery, New York, NY, USA, 321–333. <https://doi.org/10.1145/347059.347563>
- Liam O'Connor, Christine Rizkallah, Zilin Chen, Sidney Amani, Japheth Lim, Yutaka Nagashima, Thomas Sewell, Alex Hixon, Gabriele Keller, Toby Murray, et al. 2016. COGENT: certified compilation for a functional systems language. *arXiv preprint arXiv:1601.05520* (2016).
- Chris Okasaki. 1998. *Purely Functional Data Structures*. Cambridge University Press.
- Tahina Ramananandro, Antoine Delignat-Lavaud, Cédric Fournet, Nikhil Swamy, Tej Chajed, Nadim Kobeissi, and Jonathan Protzenko. 2019. Everparse: Verified Secure Zero-Copy Parsers for Authenticated Message Formats. In *Proceedings of the 28th USENIX Conference on Security Symposium* (Santa Clara, CA, USA) (*SEC'19*). USENIX Association, USA, 1465–1482.
- Marcell van Geest and Wouter Swierstra. 2017. Generic Packet Descriptions: Verified Parsing and Pretty Printing of Low-Level Data. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Type-Driven Development* (Oxford, UK) (*TyDe 2017*). Association for Computing Machinery, New York, NY, USA, 30–40. <https://doi.org/10.1145/3122975.3122979>
- Michael Vollmer, Chaitanya Koparkar, Mike Rainey, Laith Sakka, Milind Kulkarni, and Ryan R. Newton. 2019. LoCal: A Language for Programs Operating on Serialized Data. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (*PLDI 2019*). Association for Computing Machinery, New York, NY, USA, 48–62. <https://doi.org/10.1145/3314221.3314631>
- Michael Vollmer, Sarah Spall, Buddhika Chamith, Laith Sakka, Chaitanya Koparkar, Milind Kulkarni, Sam Tobin-Hochstadt, and Ryan R. Newton. 2017. Compiling Tree Transforms to Operate on Packed Representations. In *31st European Conference on Object-Oriented Programming (ECOOP 2017) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 74)*, Peter Müller (Ed.). Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 26:1–26:29. <https://doi.org/10.4230/LIPIcs.ECOOP.2017.26>
- Qianchuan Ye and Benjamin Delaware. 2019. A Verified Protocol Buffer Compiler. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Cascais, Portugal) (*CPP 2019*). Association for Computing Machinery, New York, NY, USA, 222–233. <https://doi.org/10.1145/3293880.3294105>